# CircuitVAE: Efficient and Scalable Latent Circuit Optimization

JIALIN SONG*, NVIDIA
AIDAN SWOPE†*, Harmonic
ROBERT KIRBY, NVIDIA
RAJARSHI ROY, NVIDIA
SAAD GODIL†, Hippocratic AI
JONATHAN RAIMAN, NVIDIA
BRYAN CATANZARO, NVIDIA

Automatically designing fast and space-efficient digital circuits is challenging because circuits are discrete, must exactly implement the desired logic, and are costly to simulate. We address these challenges with CircuitVAE, a search algorithm that embeds computation graphs in a continuous space and optimizes a learned surrogate of physical simulation by gradient descent. By carefully controlling overfitting of the simulation surrogate and ensuring diverse exploration, our algorithm is highly sample-efficient, yet gracefully scales to large problem instances and high sample budgets. We test CircuitVAE by designing binary adders across a large range of sizes, IO timing constraints, and sample budgets. Our method excels at designing large circuits, where other algorithms struggle: compared to reinforcement learning and genetic algorithms, CircuitVAE typically finds 64-bit adders which are smaller and faster using less than half the sample budget. We also find CircuitVAE can design state-of-the-art adders in a real-world chip, demonstrating that our method can outperform commercial tools in a realistic setting.

**ACM Reference Format:**
Jialin Song, Aidan Swope, Robert Kirby, Rajarshi Roy, Saad Godil, Jonathan Raiman, and Bryan Catanzaro. 2024. CircuitVAE: Efficient and Scalable Latent Circuit Optimization. In *61st ACM/IEEE Design Automation Conference (DAC '24), June 23–27, 2024, San Francisco, CA, USA.* ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/3649329.3656543

## 1 INTRODUCTION

As the workhorses of today's parallel processors, optimizing the design of binary adders is an important and well-studied problem. However, because the physical characteristics of a given design change as manufacturing technology improves, and because each adder in a larger design may face different constraints, classical adder designs which minimize analytical properties such as circuit depth often perform poorly in practice. More recent approaches use physical synthesis and simulation to optimize adders for real-world area, delay, and power consumption [18, 21]. Such algorithms remain expensive, though, because physical synthesis is slow, the problem is discrete, and the search space grows exponentially with the number of bits to be summed. Therefore, optimizing larger adders has generally remained intractable.

We present CircuitVAE, a highly sample-efficient algorithm for optimizing binary adders which outperforms human designs and commercial tools while requiring fewer simulations than competing approaches. CircuitVAE solves the two key challenges of this domain, discrete search and an expensive objective function, by embedding circuits in a continuous space and learning to predict the results of physical simulation. We use a $\beta$-VAE [9] to represent adders as continuous latent vectors, jointly trained with a neural cost predictor whose gradients help organize the latent space according to cost. At search time, we use gradient descent through the cost model to search for latent vectors minimizing predicted cost. We introduce two domain-agnostic improvements to the standard latent-space optimization framework. Our first, prior-regularized search, prevents search points from "overfitting" the cost predictor far from the data manifold. The second, cost-weighted sampling, helps balance quality and diversity in the explored points by initializing them from high-performing prior evaluations. Through extensive ablations, we demonstrate that these improvements enable gradient-based search to outperform Bayesian optimization in the latent space, contrary to the standard approach.

We evaluate CircuitVAE in numerous settings, both on standard benchmarks and in the real world. Across various sizes of adders, and emulating various cost tradeoffs between area and delay, we find that CircuitVAE outperforms human designs, commercial tools, and the prior state-of-the-art reinforcement learning algorithm in cost and simulation requirements. Finally, we integrate CircuitVAE into a real-world chip design workflow and show that it outperforms commercial tools in a realistic setting.

## 2 RELATED WORK

### 2.1 Machine learning for EDA

The most closely related work to ours is [18], which also optimizes parallel prefix adders with deep learning. While their reinforcement learning (RL) approach outperforms conventional tools, we show that RL is hindered by the difficulty of searching directly in the input space. In head-to-head comparison (subsection 5.2), CircuitVAE is typically more than twice as data-efficient as RL due to learning its own well-structured search space. Classical approaches to this problem include heuristic search [16] and pruning [19] methods.

Several works have explored using machine learning for other parts of the electronic design automation (EDA) process. These include [15] and [14], who use deep learning for chip placement,

*Equal Contribution
†Work done while at NVIDIA


*DAC '24, June 23–27, 2024, San Francisco, CA, USA*

ACM ISBN 979-8-4007-0601-1/24/06
https://doi.org/10.1145/3649329.3656543

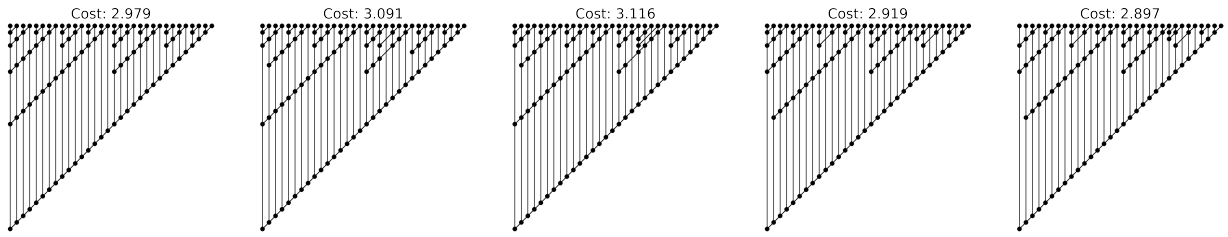


Fig. 1. A sample evolution of 32-bit adders discovered by CircuitVAE. Starting from the Sklansky structure, CircuitVAE iteratively navigates a learned latent space to produce different designs until reaching a lowest cost one on the right.

[23] who use RL to design analog rather than digital circuits, and [24] who use RL to design multipliers.

We build on the open-source EDA tools OpenROAD [2] and OpenPhySyn [1], without which this work would not have been possible.

### 2.2 Latent-space optimization

CircuitVAE employs latent-space optimization (LSO), a method which has recently become popular for black-box optimization, most notably in the field of chemical design [8, 10, 22]. LSO consists of learning a latent-variable generative model over input structures, together with a neural predictor of the cost function which serves to shape the latent space and may be used for optimization. The latent space acts as a learned continuous search space, typically grouping semantically similar inputs and enabling continuous search algorithms. While many improvements to this scheme have been recently proposed, we build on the framework of [22], which interleaves optimization with retraining the generative model on new data.

While some LSO techniques optimize by gradient descent through the cost predictor, recently Bayesian optimization (BO) has been the preferred approach [10]. In this work, we demonstrate that naive gradient descent suffers from over-optimizing the cost predictor far from the data manifold, yielding points with low predicted costs but high actual costs. However, we introduce two techniques to address this problem and show that, once appropriately regularized, gradient descent can outperform BO by a wide margin.

## 3 BACKGROUND

Many kinds of circuits, notably including binary adders, can be compactly represented as *prefix graphs*, which describe the pattern of carries within the circuit. Prefix graphs compactly represent a circuit's design in terms of carry *generation* and *propagation* [4]. Each bit span `i:j` is associated with a generate bit $g_{i,j}$ and a propagate bit $p_{i,j}$. For all $i = 1 \dots N$, computing the input bits $g_{i,i}$ and $p_{i,i}$ is straightforward; furthermore, given the output bits $(g_{1,1}; p_{1,1}) \dots (g_{N,1}, p_{N,1})$, computing the final carries and summand is easy. Intermediate values may be computed recursively: $(g_{i,j}; p_{i,j}) = (g_{i,x}; p_{i,x}) \circ (g_{x-1,j}; p_{x-1,j})$ where $i \geq x > j$ and $\circ$ is the carry operator Brent and Kung describe. A prefix graph is exactly a tree determining the association order of $(g_{i,i}; p_{i,i}) \circ \dots \circ (g_{1,1}; p_{1,1})$ for each $i = 1, \dots, N$.

The design space of prefix graphs is large—$O(2^{N^2})$ for $N$-bit designs—and expresses tradeoffs between circuit area and delay, the two main desiderata. For example, the ripple-carry structure represents schoolbook addition, computing carries one bit at a time, and has the lowest possible area but is relatively slow. Faster adders such as Sklansky [20] and Kogge-Stone [13] compute redundant carry bits, which enables some parallelism at the cost of area. While regular structures minimizing analytical properties like graph depth and connectivity are well-known, the actual delay of a fully synthesized and laid-out circuit depends in a complicated way on many other physical factors, so designing these circuits remains an important industrial problem.

Because real-world designs may have different requirements for area and latency, we define a scalar cost function $f(x) = \omega \cdot \text{delay}(x) + (1 - \omega) \cdot \text{area}(x)$. We call $\omega$ the *delay weight*, a hyperparameter trading off these competing goals. In the cost function, we measured a circuit's total area in square microns divided by 100 and the delay of its longest ("critical") path in nanoseconds multiplied by 10, as we found this yielded smooth changes in optimization as $\omega$ was swept from 0 to 1. To ensure our results generalize, we conducted our experiments for $\omega \in \{0.33, 0.66, 0.95\}$ and for 32-, and 64-bit adders, as well as an experiment for designing 26-bit gray-to-binary converters with $\omega = 0.6$.

Prefix graphs are translated into physical circuits through cell mapping (which translates the logical graph into a list of electrical components with a lookup table), physical synthesis, and layout. In this work we experiment with binary adders and gray-to-binary converters [5], but the algorithm we describe could straightforwardly apply to any parallel prefix circuit by altering the cell mapping process.

## 4 CIRCUITVAE

In this section, we describe our CircuitVAE algorithm for optimizing prefix adders. Figure 2 gives an overview of the algorithm. In subsection 4.1, we describe how to train CircuitVAE by combining the standard VAE training objective with a cost predictor loss. In subsection 4.2, we describe how to search in the latent space of CircuitVAE, including our proposed prior-regularized search and cost-weighted sampling techniques.

### 4.1 Training

We denote by $\mathcal{X}$ the discrete search space for all $N$-bit adders. Optimizing over $\mathcal{X}$ is challenging: computation graphs with many nodes or edges in common may not have similar costs because removing one node is sufficient to change the critical path. Therefore, we

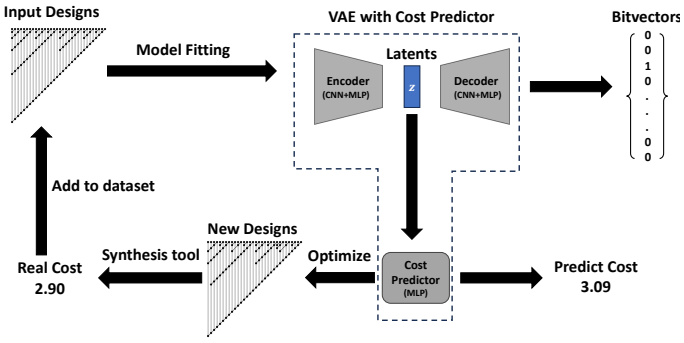


Fig. 2. The CircuitVAE algorithm flowchart. We first fit a VAE equipped with a cost predictor on an input dataset. We use prior-regularized search (subsection 4.2) to optimize the cost predictor to generate new designs. We repeat this loop multiple times to gradually discover better designs.

embed $\mathcal{X}$ into a continuous latent space $\mathcal{Z}$ with a VAE augmented with a cost prediction head.

Concretely, we learn an encoding function $q_\phi(z|x)$ that encodes an input $x$ to a distribution over $\mathcal{Z}$, and a decoding function $p_\theta(x|z)$ that decodes a latent variable $z$ to a distribution over $\mathcal{X}$. We optimize the parameters $\theta$ and $\phi$ by maximizing the evidence lower-bound (ELBO) [12]: $\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)] - D_{KL}(q_\phi(z|x)||p_\theta(z))$, where $D_{KL}$ is the Kullback–Leibler divergence between two distributions. Our prior $p_\theta(z)$ is a diagonal unit Gaussian. To balance adherence to the prior with other training objectives, we use a $\beta$-VAE [3, 9]. Our training loss is:

$$\mathcal{L}_{\theta,\phi}(x) = \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)] - \beta D_{KL}(q_\phi(z|x)||p_\theta(z)) \quad (1)$$

Given a dataset of prefix adders and their costs $\mathcal{D} = \{(x_i, c_i)\}_{i=1}^n$, we can fit the VAE parameters by maximizing $\sum_{i=1}^n \mathcal{L}_{\theta,\phi}(x_i)$ via gradient descent using the reparameterization trick [12].

In addition to learning the encoder and decoder, we also learn a cost predictor model $f_\pi : \mathcal{Z} \to \mathbb{R}$. For a datapoint $(x, c)$, we first map $x$ to a latent space distribution $q_\phi(z|x)$, next we sample a latent variable $z \sim q_\phi(z|x)$ (again using the reparametrization trick), and finally we predict its cost $f_\pi(z)$. Our cost prediction loss is $\mathcal{L}_\pi(x, c) = (f_\pi(z) - c)^2$. The cost predictor enables optimization, but it also helps shape the latent space: observe that if two circuits $x_1, x_2$ with very different costs have overlapping posteriors $q_\phi(\cdot \mid x_1), q_\phi(\cdot \mid x_2)$, the cost predictor will fail to distinguish them. Therefore, the training loss is minimized when circuits with similar costs are grouped together, which aids optimization.

Following [22], we reweight datapoints according to their cost to give promising points more volume in latent space. Specifically, the weight of a datapoint $(x, c) \in \mathcal{D}$ is

$$w(x; \mathcal{D}, k) \propto \frac{1}{kn + \text{rank}_{\mathcal{D}}(x)}, \text{rank}_{\mathcal{D}}(x) = |\{x_i : c_i < c,\ (x_i, c_i) \in \mathcal{D}\}|\ , \quad (2)$$

where $k$ is a hyperparameter controlling the relative weights among the datapoints. For simplicity, we use $w_i(\mathcal{D})$ to denote the normalized weight for a datapoint $(x_i, c_i)$. Note that we need to recompute these weights after acquiring new datapoints because of the dependency on $\mathcal{D}$.

Finally, we can put the two losses together to obtain the overall loss objective

$$\mathcal{L}_{\theta,\phi,\pi}(\mathcal{D}) = \sum_{i=1}^n w_i(\mathcal{D})\mathcal{L}_{\theta,\phi}(x_i) + \lambda \mathcal{L}_\pi(x_i, c_i) \quad (3)$$

where $\lambda \in \mathbb{R}^+$ is a hyperparameter to balance the VAE training loss and the cost prediction loss. In all experiments, we set $\beta = 0.01$, $\lambda = 10.0$, and $k = 0.001$, and optimize $\mathcal{L}_{\theta,\phi,\pi}$ with Adam [11].

**Algorithm 1** CircuitVAE

**Input**: $\mathcal{D}_0$ initial dataset; $f$ a blackbox function available for queries; $M$ the number of data acquisition rounds; $m$ the number of parallel latent search; $T$ the number of gradient descent steps for latent space optimization; $t$ the interval to capture latents during optimization
$\mathcal{D} \leftarrow \mathcal{D}_0$
**for** $i = 1...M$ **do**
  Compute sample weights for $\mathcal{D}$ (Equation 2)
  Fit parameters $(\theta, \phi, \pi)$ of VAE with the cost predictor on $\mathcal{D}$ with the weighted training objective (Equation 3)
  Sample $m$ points from $\mathcal{D}$ proportional to sample weights
  Sample $m$ initial latents with $q_\phi$
  Optimize $g(z)$ (Equation 4) with gradient descent from the initial latents for $T$ steps and capture a set of latents $Z_i$ along the optimization trajectory after every $t$ gradient steps
  Sample a new set of $X_i$ by decoding $Z_i$ through $p_\theta$
  Query $f$ on $\mathcal{X}_t$ to obtain $\mathcal{D}_i$
  $\mathcal{D} \leftarrow \mathcal{D} \cup \mathcal{D}_i$
**end for**
**return** the lowest cost point in $\mathcal{D}$

## 4.2 Optimization

Once we fit the parameters for the VAE with the cost predictor, we can choose new designs to query by minimizing the predicted costs with $f_\pi$. In our experiments, we instantiate $f_\pi$ with a feed-forward neural network and perform gradient descent directly in the latent space by differentiating through $f_\pi$ with respect to its inputs.

Naively optimizing the predicted cost without constraints yields poor results [6, 7, 17] because the cost predictor is only accurate near regions where training examples are available. We propose *prior regularization*, a means of softly constraining optimized latents to stay near the origin where the majority of the dataset lies. We optimize latents according to a linear combination of predicted cost and prior log-probability:

$$g(z) = f_\pi(z) - \gamma \log p_\theta(z) \quad (4)$$

where $\gamma$ is a hyperparameter to control the strength of the prior regularization. While [22] propose constraining search to a box around the origin for the same reason, we note that in high-dimensional space a box has exponentially many corners, and so a box large enough to contain most of the data mass is likely to have many uninhabited regions.

Table 1. Detailed comparison of methods in the 64-bit, high-budget setting

| $\omega$ | Alg. | Cost | Area ($\mu m^2$) | Delay (ns) | VAE speedup |
|---|---|---|---|---|---|
| 0.33 | VAE | **4.54** (4.52 - 4.55) | **449** (446 - 452) | **0.465** (0.446 - 0.468) | – |
| | GA | 4.65 (4.59 - 4.67) | 463 (458 - 465) | 0.480 (0.451 - 0.487) | 3.03 (2.15 - 3.45) |
| | RL | 4.72 (4.71 - 4.72) | 471 (464 - 482) | 0.473 (0.462 - 0.474) | 3.36 (2.46 - 3.48) |
| | BO | 4.94 (4.80 - 4.97) | 470 (468 - 475) | 0.536 (0.512 - 0.542) | 7.34 (5.99 - 8.50) |
| 0.66 | VAE | **4.31** (4.31 - 4.38) | **572** (572 - 579) | **0.360** (0.359 - 0.363) | – |
| | GA | 4.44 (4.40 - 4.45) | 598 (587 - 606) | 0.363 (0.362 - 0.364) | 1.88 (1.65 - 1.93) |
| | RL | 4.63 (4.63 - 4.64) | 650 (650 - 656) | 0.368 (0.364 - 0.369) | 2.69 (2.18 - 4.88) |
| | BO | 4.66 (4.64 - 4.72) | 635 (607 - 654) | 0.388 (0.369 - 0.397) | 3.15 (1.22 - 10.36) |
| 0.95 | VAE | **3.58** (3.58 - 3.59) | 860 (834 - 902) | **0.333** (0.330 - 0.333) | – |
| | GA | 3.60 (3.60 - 3.61) | 868 (855 - 872) | 0.334 (0.334 - 0.335) | 2.30 (2.21 - 5.49) |
| | RL | 3.70 (3.69 - 3.70) | **801** (800 - 803) | 0.347 (0.347 - 0.347) | 3.26 (3.18 - 5.82) |
| | BO | 3.72 (3.71 - 3.72) | 827 (806 - 836) | 0.347 (0.347 - 0.350) | 2.47 (1.96 - 2.48) |

To balance quality and diversity of our samples, we also initialize the starting latent variables close to valid and high-performing circuits by a cost-weighted sampling method. Specifically, we sample circuits from the current dataset (Line 6 in Algorithm 1) proportional to their datapoint weights (Equation 2). For a sample design $x$, we obtain an initial latent variable $z_0$ from the posterior $q_\phi(z|x)$ (Line 7) and the gradient descent on $g(z)$ starts at $z_0$. This procedure ensures that latents are initialized in high-probability and low-cost regions, while being diverse enough to provide good training data for the next round.

We perform gradient descent for a fixed number of steps and capture the latent variable values after every $t$ steps to get $z_t, z_{2t}$, etc. (Line 8). For each $z_t$, we decode to a distribution over $\mathcal{X}$ with $p_\theta(x|z_t)$ and sample a design $x$ to query its cost $c$ (Line 9 and 10). CircuitVAE (Algorithm 1) repeats the training and optimization loop multiple times. In practice, we can parallelize latent space gradient descent (Line 8) to further accelerate the search. Figure 1 presents an example optimization trajectory for CircuitVAE.

## 5 EXPERIMENTS

### 5.1 Training and evaluation details

We evaluated circuits following [18]. Prefix graphs generated by CircuitVAE are first legalized by inserting missing parents of existing nodes—this may be considered part of the objective function, so our cost predictor learns to infer the same value for equivalent circuits. We then compile the legalized graph into a netlist using the 45-nanometer Nangate45 cell library [2] and then physically synthesize the circuit using OpenPhySyn [1]. $N$-bit prefix graphs are represented with in a $N \times N$ matrix as in [18]. Our encoder and decoder were each $\sim$ 1M-parameter CNNs autoencoding the computation graph with a 2-layer MLP as the cost predictor. We represent circuits using the grid format described by [18]. All experiments used one A100 GPU and 24 CPU cores.

### 5.2 Comparing search algorithms

We compared CircuitVAE to three alternative search algorithms on the task of optimizing adders across a range of simulation budgets. Our primary baseline is PrefixRL ("RL"), the prior state-of-the-art reported by [18]. We also compared against a genetic algorithm ("GA") directly optimizing a bitvector representation of the circuit; we used the first few generations of GA as the initial data to train CircuitVAE. Finally, we compared against a variant of CircuitVAE which employs Bayesian optimization (BO) in the latent space, a practice which has become common [22].

Our results are summarized in Figure 3: in all settings, CircuitVAE outperforms all other methods at every budget. The 64-bit setting (the most difficult) is detailed in Table 1, which lists the cost, area, and delay of the best single adder found by each method in less than 70,000 simulations. We also report the "VAE speedup", the simulation budget for each method to produce its best adder divided by the simulation budget for CircuitVAE to obtain an equivalent or better circuit. In almost all cases, we find that CircuitVAE requires less than half the simulations to obtain equal performance, and in many cases less than one-third.

We repeated this experiment for bitwidths in {32, 64} and delay weights in {0.33, 0.66, 0.95}. For CircuitVAE and Bayesian experiments, we launched runs with approximately 1,000, 5,000, 10,000, and 30,000 initial datapoints and grouped these runs into a single curve to report performance across a range of budgets; the initial simulations required to build the dataset were counted against these methods' budgets. We ran each experiment with five different random seeds and independently collected initial datasets, and report the median and interquartile ranges across these runs.

We found gradient-based search outperforms latent Bayesian search in all settings. We hypothesize that our higher-capacity neural cost predictor can learn more from large datasets than the Bayesian surrogate model, and that mitigating overoptimization with prior-regularized search enables quickly identifying promising candidates.

### 5.3 Ablations

We ablated each of the components of CircuitVAE to understand their individual contributions. All of these experiments were conducted on 32-bit adders, with a delay weight of 0.66 and the largest initial dataset. In Figure 4, we tested:

- Removing data reweighting [22], which leads training to get stuck when new datapoints have a negligible impact on the overall distribution.
- Replacing the cost-weighted latent distribution with the prior or the latent encoding of Sklansky. Starting the search from a good adder (Sklansky) outperforms sampling from the prior, but both underperform our adaptive initialization.

In Figure 5, we examined the ability to control search by controlling the prior regularization term $\gamma$. At low values of $\gamma$ (blue and orange), latent trajectories quickly exit the region around the training data (gray) and overfit the cost predictor, yielding much higher costs than the model predicts. At higher values (green and red), trajectories stay near the origin, which prevents overfitting but limits exploration and sample diversity. We found the best results from sampling values of $\gamma$ per latent trajectory log-uniformly between 0.01 and 0.1, and used this setting for all other experiments.

### 5.4 Designing real-world circuits

To evaluate CircuitVAE in a more realistic setting, we tried using it in place of a commercial tool in a real-world datapath design. We used CircuitVAE to design 31-bit adders at delay weights in {0.3, 0.6,

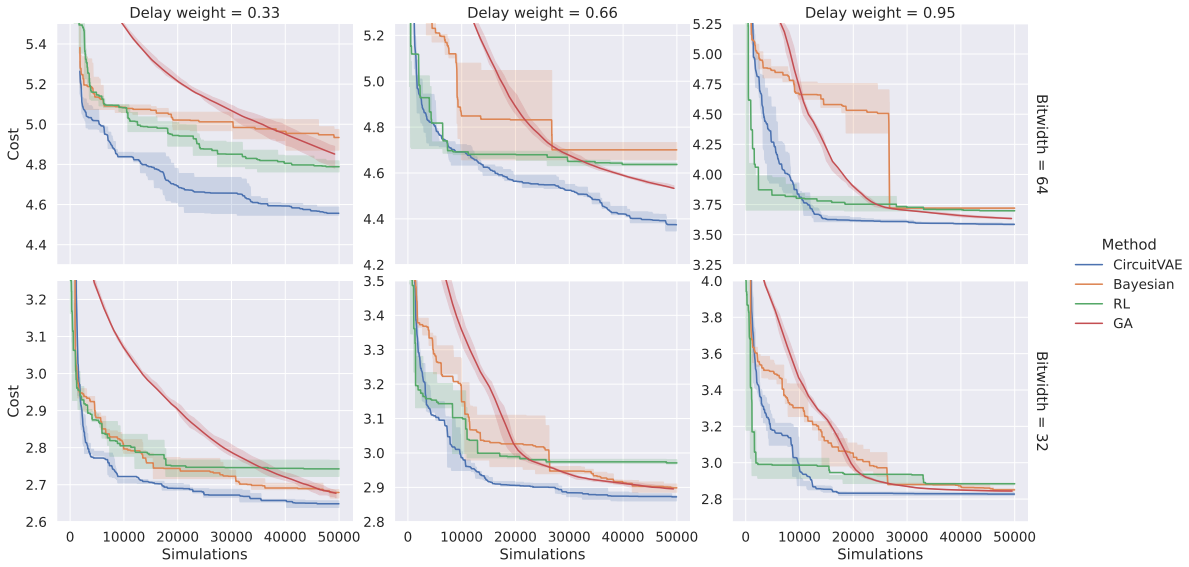


Fig. 3. Curves of circuit cost (lower is better) vs simulation budget across a range of circuit sizes (rows) and timing constraints (columns). CircuitVAE consistently achieves lower costs at fewer simulations. The first and second row are 64-bit and 32-bit design tasks, respectively.

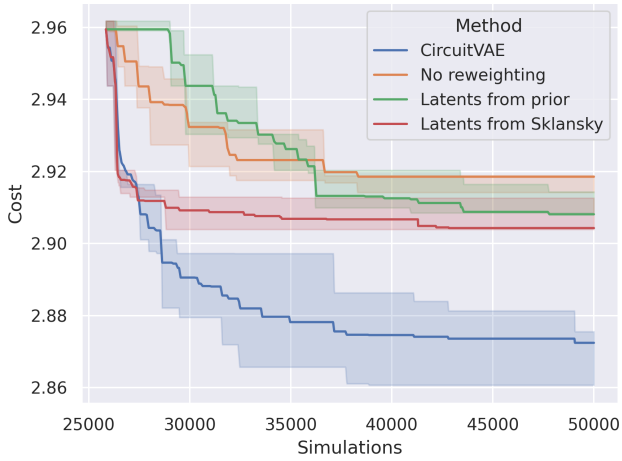


Fig. 4. Ablating search and training methods.

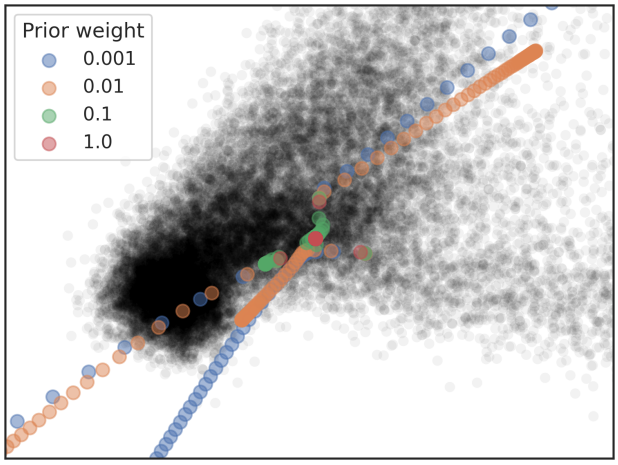


Fig. 5. The effect of changing prior weight $\gamma$ on latent search trajectories. Low values of $\gamma$ result in trajectories ending far away from training data.

0.95} using OpenPhySyn as described above; however, we generated netlists with a proprietary 8nm cell library, and used bit input and output timings captured from a complete datapath. Then, we synthesized the most promising designs using a commercial design tool. Note the domain gap in the cost function between training and evaluation: the commercial tool makes different choices with respect to netlist buffering, gate sizing, cell placement, etc. Nevertheless, as Figure 6 shows, we managed to Pareto-dominate both the design tool's provided adders and common human-designed adders.

### 5.5 Designing gray-to-binary converts

To showcase the generality of our CircuitVAE framework, we also designed a 26-bit gray-to-binary converter [5]. For this experiment, we target a delay weight of 0.6 and use the Nangate45 cell library to compile netlists. Figure 7 shows comparison results with the same set of baseline methods as in subsection 5.2. CircuitVAE outperforms all baselines as well in this design task.

Finally, Figure 8 shows the best designs for the gray-to-binary converter and a 32-bit adder. Although both design tasks targeted similar delay weights, there are significant structural differences in best designs discovered by CircuitVAE. They demonstrate that CircuitVAE can flexibily adapt to different circuit types.

## 6 CONCLUSION

In this work, we demonstrated that CircuitVAE can efficiently optimize binary adders and gray-to-binary converters, even in the

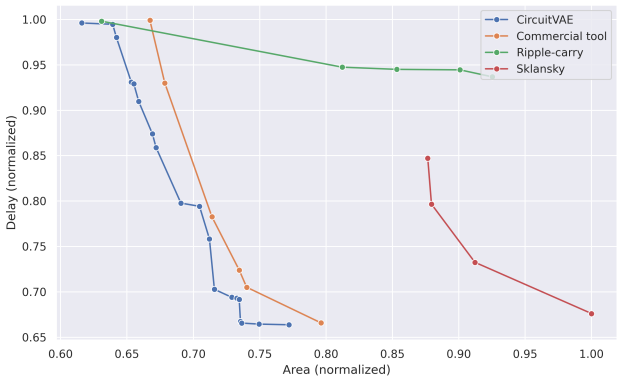


Fig. 6. Comparing the area-delay Pareto frontiers of 8nm circuits in a realistic setting. CircuitVAE's designs Pareto-dominate both human-designed circuits and a commercial design tool.

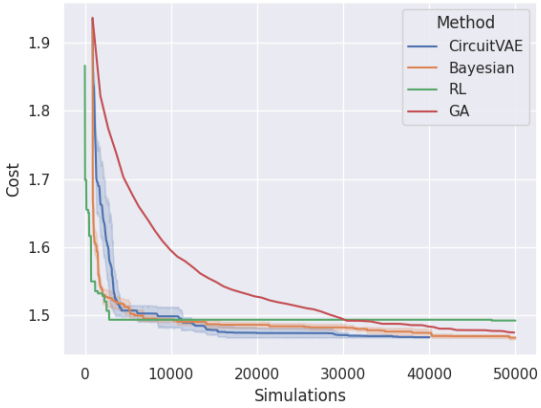


Fig. 7. Curves of circuit cost (lower is better) vs simulation budget for gray-to-binary converters. CircuitVAE consistently achieves lower costs at fewer simulations.

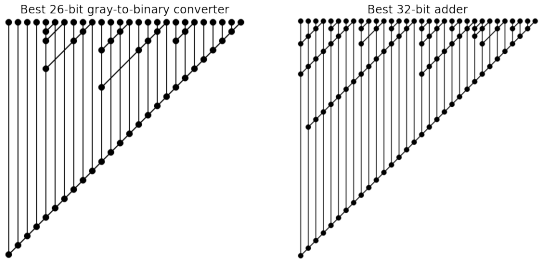


Fig. 8. Best designs for gray-to-binary converter and adder. Their structural differences validate the two tasks are fundamentally different.

difficult cases of large circuits and tight timing constraints. However, the authors optimistically believe that the impact of this work extends these two classes of circuits. Our method may be applied unchanged to optimize other prefix computations, such as leading zero detectors; by replacing the prefix graph with another data structure, one might also optimize multipliers or other circuits. We hope to address these exciting possibilities in future work.

## REFERENCES


[1] Ahmed Agiza and Sherief Reda. 2020. OpenPhySyn: An Open-Source Physical Synthesis Optimization Toolkit.

[2] T Ajayi, D Blaauw, TB Chan, CK Cheng, VA Chhabria, DK Choo, M Coltella, S Dobre, R Dreslinski, M Fogaça, et al. 2019. OpenROAD: Toward a Self-Driving, Open-Source Digital Layout Implementation Tool Chain. *Proc. GOMACTECH* (2019), 1105–1110.

[3] Alican Bozkurt, Babak Esmaeili, Jean-Baptiste Tristan, Dana Brooks, Jennifer Dy, and Jan-Willem van de Meent. 2021. Rate-regularization and generalization in variational autoencoders. In *International Conference on Artificial Intelligence and Statistics*. PMLR, 3880–3888.

[4] R.P. Brent and H.Y. Kung. 1982. A regular layout for parallel adders. *IEEE Transactions on Computer* C-31 (1982), 260–264.

[5] Robert W Doran. 2007. *The gray code*. Technical Report. Citeseer.

[6] Rafael Gómez-Bombarelli, Jennifer N Wei, David Duvenaud, José Miguel Hernández-Lobato, Benjamín Sánchez-Lengeling, Dennis Sheberla, Jorge Aguilera-Iparraguirre, Timothy D Hirzel, Ryan P Adams, and Alán Aspuru-Guzik. 2018. Automatic chemical design using a data-driven continuous representation of molecules. *ACS central science* 4, 2 (2018), 268–276.

[7] Ryan-Rhys Griffiths and José Miguel Hernández-Lobato. 2020. Constrained Bayesian optimization for automatic chemical design using variational autoencoders. *Chemical science* 11, 2 (2020), 577–586.

[8] Rafael Gómez-Bombarelli, Jennifer N. Wei, David Duvenaud, José Miguel Hernández-Lobato, Benjamín Sánchez-Lengeling, Dennis Sheberla, Jorge Aguilera-Iparraguirre, Timothy D. Hirzel, Ryan P. Adams, and Alán Aspuru-Guzik. 2018. Automatic Chemical Design Using a Data-Driven Continuous Representation of Molecules. *ACS Central Science* 4, 2 (2018), 268–276.

[9] Irina Higgins, Loic Matthey, Arka Pal, Christopher Burgess, Xavier Glorot, Matthew Botvinick, Shakir Mohamed, and Alexander Lerchner. 2017. beta-vae: Learning basic visual concepts with a constrained variational framework. In *International conference on learning representations*.

[10] Wengong Jin, Regina Barzilay, and Tommi Jaakkola. 2018. Junction tree variational autoencoder for molecular graph generation. In *International conference on machine learning*. PMLR, 2323–2332.

[11] Diederik P Kingma and Jimmy Ba. 2014. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980* (2014).

[12] Diederik P Kingma and Max Welling. 2013. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114* (2013).

[13] Peter M. Kogge and Harold S. Stone. 1973. A Parallel Algorithm for the Efficient Solution of a General Class of Recurrence Equations. *IEEE Trans. Comput.* 22, 8 (1973), 786–793.

[14] Yibo Lin, Zixuan Jiang, Jiaqi Gu, Wuxi Li, Shounak Dhar, Haoxing Ren, Brucek Khailany, and David Z. Pan. 2020. DREAMPlace: Deep Learning Toolkit-Enabled GPU Acceleration for Modern VLSI Placement. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 40 (2020), 748–761.

[15] Azalia Mirhoseini, Anna Goldie, Mustafa Yazgan, Joe W. J. Jiang, Ebrahim M. Songhori, Shen Wang, Young-Joon Lee, Eric Johnson, Omkar Pathak, Sungmin Bae, Azade Nazi, Jiwoo Pak, Andy Tong, Kavya Srinivasa, William Hang, Emre Tuncer, Anand Babu, Quoc V. Le, James Laudon, Richard Ho, Roger Carpenter, and Jeff Dean. 2020. Chip Placement with Deep Reinforcement Learning. *CoRR* abs/2004.10746 (2020). arXiv:2004.10746 https://arxiv.org/abs/2004.10746

[16] Takayuki Moto and Mineo Kaneko. 2018. Prefix Sequence: Optimization of Parallel Prefix Adders using Simulated Annealing. In *2018 IEEE International Symposium on Circuits and Systems (ISCAS)*. 1–5.

[17] Anh Nguyen, Alexey Dosovitskiy, Jason Yosinski, Thomas Brox, and Jeff Clune. 2016. Synthesizing the preferred inputs for neurons in neural networks via deep generator networks. *Advances in neural information processing systems* 29 (2016).

[18] Rajarshi Roy, Jonathan Raiman, Neel Kant, Ilyas Elkin, Robert Kirby, Michael Siu, Stuart Oberman, Saad Godil, and Bryan Catanzaro. 2021. PrefixRL: Optimization of Parallel Prefix Circuits using Deep Reinforcement Learning. In *2021 58th ACM/IEEE Design Automation Conference (DAC)*. 853–858.

[19] Subhendu Roy, Mihir R. Choudhury, Ruchir Puri, and David Z. Pan. 2014. Towards Optimal Performance-Area Trade-Off in Adders by Synthesis of Parallel Prefix Structures. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 33, 10 (2014), 1517–1530.

[20] J. Sklansky. 1960. Conditional-Sum Addition Logic. *Ire Transactions on Electronic Computers* 9, 2 (1960), 226–231.

[21] Jialin Song, Rajarshi Roy, Jonathan Raiman, Robert Kirby, Neel Kant, Saad Godil, and Bryan Catanzaro. 2022. Multi-objective Reinforcement Learning with Adaptive Pareto Reset for Prefix Adder Design. *Workshop on ML for Systems at NeurIPS* (2022).

[22] Austin Tripp, Erik Daxberger, and José Miguel Hernández-Lobato. 2020. Sample-efficient optimization in the latent space of deep generative models via weighted retraining. *Advances in Neural Information Processing Systems* 33 (2020), 11259–11272.

[23] Hanrui Wang, Jiacheng Yang, Hae-Seung Lee, and Song Han. 2020. Learning to Design Circuits. arXiv:1812.02734 [cs.LG]

[24] Dongsheng Zuo, Yikang Ouyang, and Yuzhe Ma. 2023. RL-MUL: Multiplier Design Optimization with Deep Reinforcement Learning. In *2023 60th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1–6.